

DBMS PROJECT DEFENSE · 2026–2027

Courier Parcel Booking & Tracking Management System

Design and Implementation of a Database Management System

Dara Keerthana(25WU0102125)

PROGRAM · B.Tech

BRANCH · AI & ML

YEAR · II Year

COURSE · Database Management
Systems

Woxsen University, Hyderabad

Academic Year 2026–2027



A web-based DBMS for every parcel's journey

The Courier Parcel Booking and Tracking Management System is a web application that manages courier operations end to end — from booking a parcel to confirming its delivery.

- Centralized storage of all courier data
- Relational structure across 13 linked tables
- Unique tracking number for every parcel
- Bookings, scans, deliveries and payments in one place
- Accurate records with enforced data integrity

13

relational tables powering one
centralized courier database

02 · PROBLEM STATEMENT

Manual courier records break down at scale

01

Manual record maintenance is slow and error-prone

02

Data scattered across registers and files

03

Tracking a parcel is complex and time-consuming

04

Delivery attempts are not recorded properly

05

Reports and summaries are hard to generate

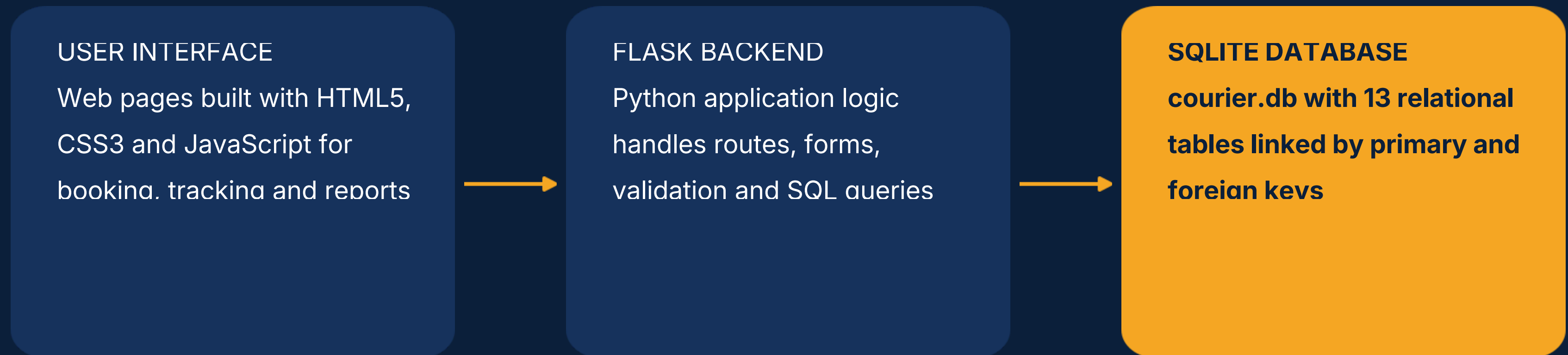
PROPOSED SOLUTION

A centralized relational database that manages the entire parcel lifecycle — from booking to delivery.

What this project sets out to achieve

- Design a well-structured relational database
- Manage customer records efficiently
- Handle parcel details and unique tracking
- Process parcel bookings across branches
- Record deliveries and delivery attempts
- Calculate charges and track payments
- Perform SQL query operations on live data
- Maintain data integrity with constraints
- Provide a user-friendly web interface

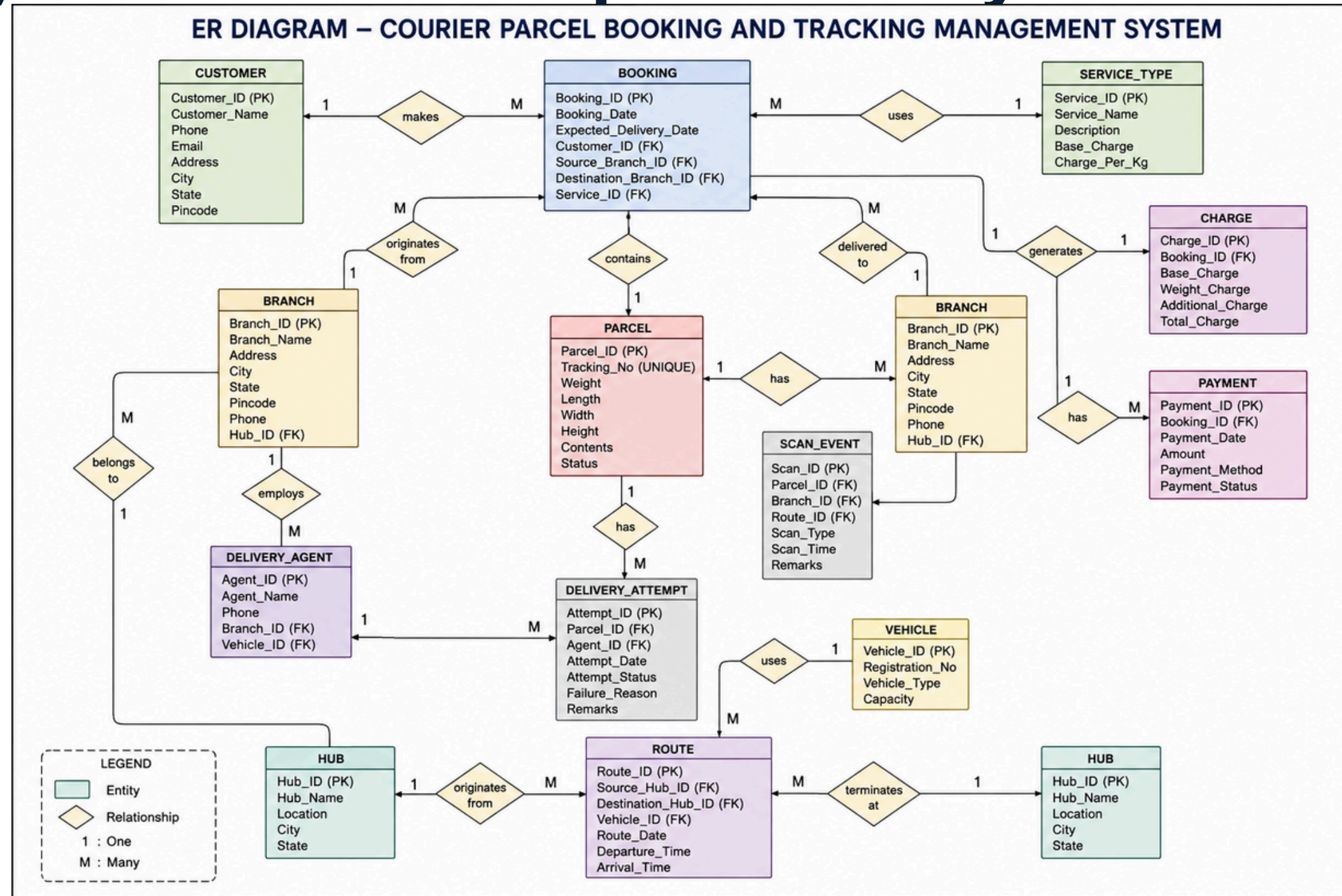
Three tiers, one clean data flow



TECHNOLOGY STACK

HTML5 · CSS3 · JavaScript · Python · Flask · SQLite · Replit · GitHub

13 entities, one connected parcel story



Normalized to 3NF, keyed for consistency

```
CUSTOMER (customer_id PK, name, phone UNIQUE, email,  
address)  
BOOKING (booking_id PK, customer_id FK, branch_id FK,  
service_type_id FK, booking_date)  
PARCEL (parcel_id PK, booking_id FK, tracking_no UNIQUE,  
weight, status)  
PAYMENT (payment_id PK, charge_id FK, amount, mode, paid_on)
```

1NF — atomic values, no repeating groups

2NF — every attribute depends on the
whole primary key

3NF — no transitive dependencies; each
fact stored once

Result: less redundancy, stronger integrity
and consistent data

Inside courier.db

13 tables 50 customers 120 parcels 120 bookings 300+ scan events

Sample data loaded across all tables for realistic testing of queries and reports

SQL OPERATIONS

CREATE • INSERT • SELECT
UPDATE • DELETE • JOIN

INTEGRITY CONSTRAINTS

- PRIMARY KEY and FOREIGN KEY links
- NOT NULL for required fields
- UNIQUE tracking numbers
- CHECK on weight, amount, status
- AUTOINCREMENT identifiers

08 · KEY QUERIES & OUTPUTS

Queries that answer real questions

-- Booking details

SELECT b.booking_id, c.name, p.tracking_no, s.name

FROM booking b

JOIN customer c ON b.customer_id = c.customer_id

JOIN parcel p ON p.booking_id = b.booking_id

JOIN service_type s ON b.service_type_id = s.service_type_id;

-- Parcel tracking history

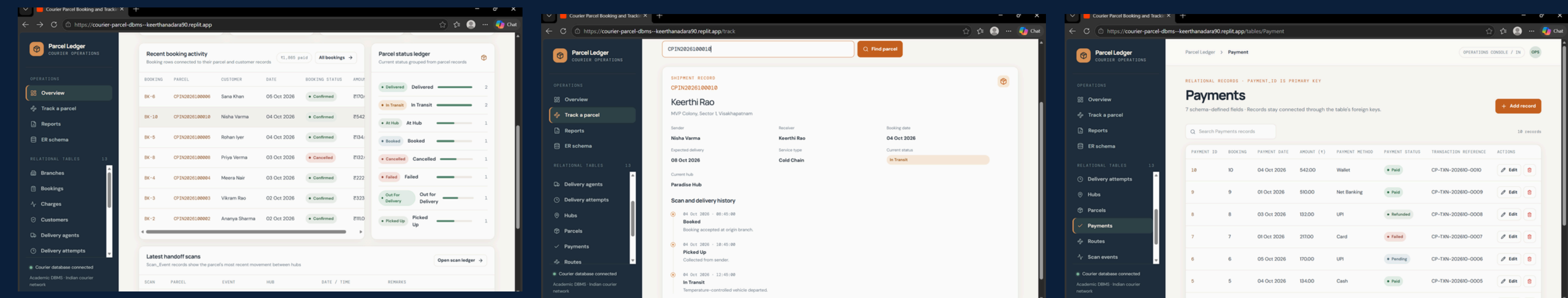
SELECT p.tracking_no, e.status, h.hub_name, e.scan_time

FROM scan_event e

JOIN parcel p ON e.parcel_id = p.parcel_id

JOIN hub h ON e.hub_id = h.hub_id

WHERE p.tracking_no = 'CPX1024' ORDER BY e.scan_time;



Live outputs: booking details · parcel tracking history · payment summary

Full CRUD, one friendly web app

- Customer management
- Parcel management
- Booking management
- Parcel tracking by number
- Delivery attempts
- Reports
- Charges and payments

